2. Multimodal Visual Understanding (Text Version)

1. Course Content

1. Learn the large model's multimodal visual perception, image semantic understanding, and logical conditional judgment capabilities;
2. Learn advanced applications such as issuing visual queries to the large model through the text terminal, summarizing the functions of environmental objects, and executing conditional actions (nodding/shaking the head);

---

2. Preparation

2.1 Content Description

> This lesson applies to controllers such as Raspberry Pi 5, Jetson Orin NANO, and D-Robotics RDK X5:
>
> - **Raspberry Pi 5**: You must first enter the Docker container before running ROS 2 related commands. Refer to [3.Enter Docker (For Raspberry Pi)] in the [Robot Configuration and Operation Guide];
> - **Orin / RDK X5**: Run commands directly in the onboard terminal; there is no need to enter Docker.

2.2 Start the Local Dify Large-Model Environment

Enter the following command in the host terminal to start the Dify service:

```
sh ~/bringup_dify.sh
```



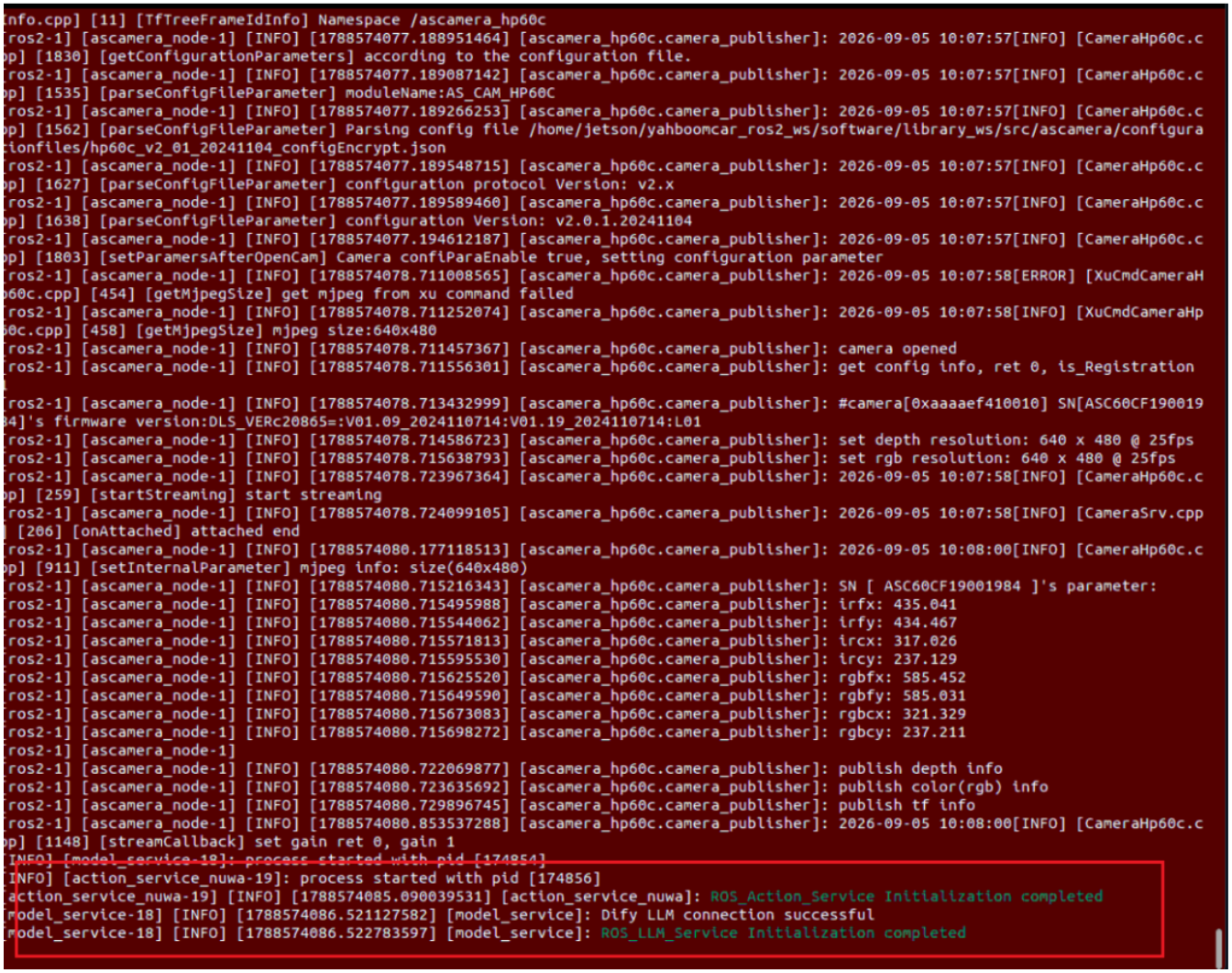
3. Running Examples

3.1 Start the Embodied Intelligence Master Control Service

Enter the following command in the onboard terminal to start the text-mode large-model embodied control system:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

After the system finishes initializing, the terminal prints the ready log:

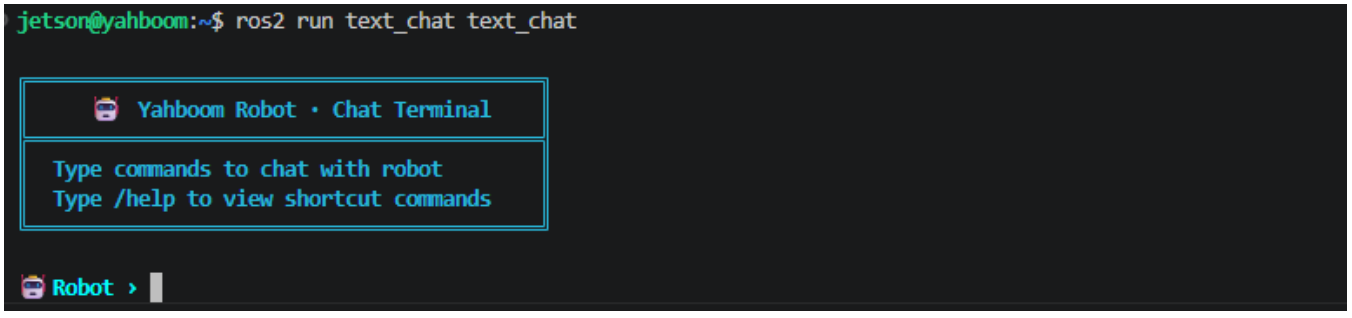


3.2 Start the Text Interaction Control Terminal

Use **remote SSH** to open a terminal for text interaction:

```
ros2 run text_chat text_chat
```

After startup, the terminal enters the real-time text interaction interface, waiting for the user to enter commands:



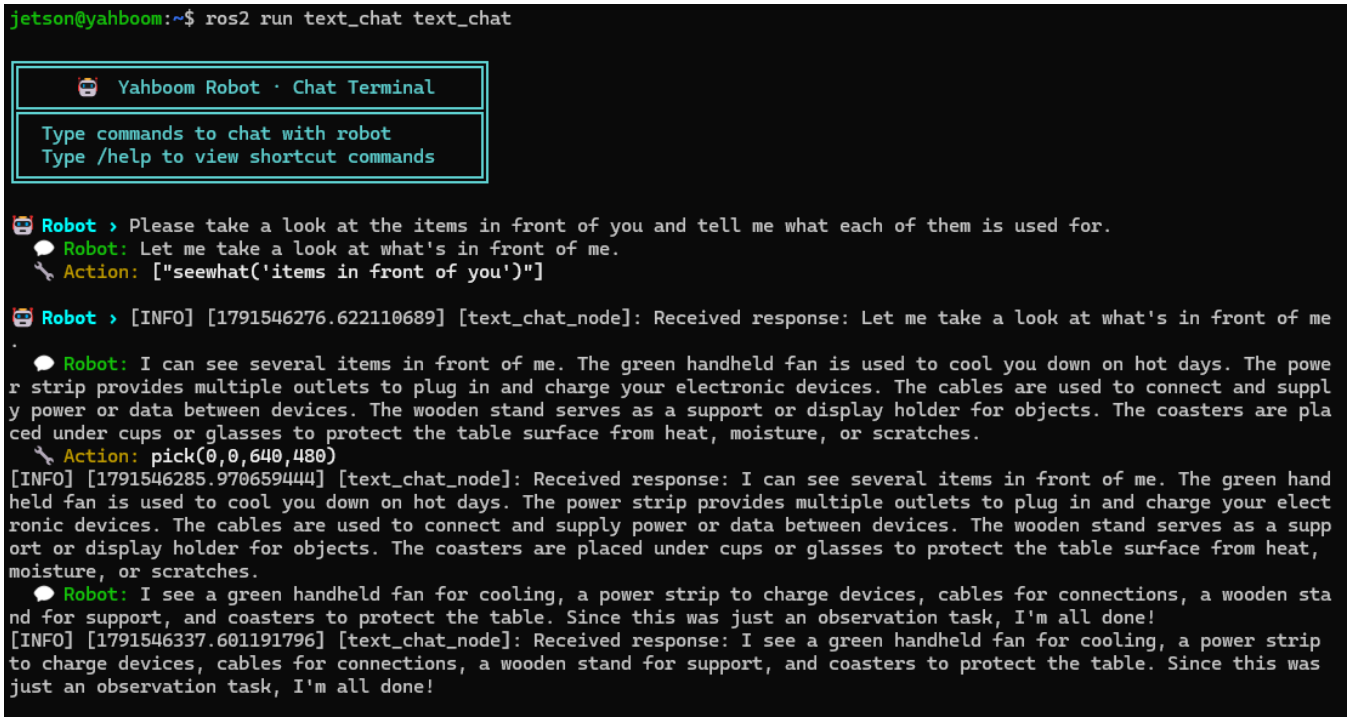
> **Tip**
>
> **Chinese Input Tip**:
>
> - It is recommended to connect from a computer using an SSH remote tool (such as MobaXterm or VSCode Terminal), so that Chinese can be typed seamlessly.

3.3 Classic Test Cases (Visual Perception and Conditional Following)

Case 1: Multimodal Recognition of Environmental Objects and Description of Their Functions

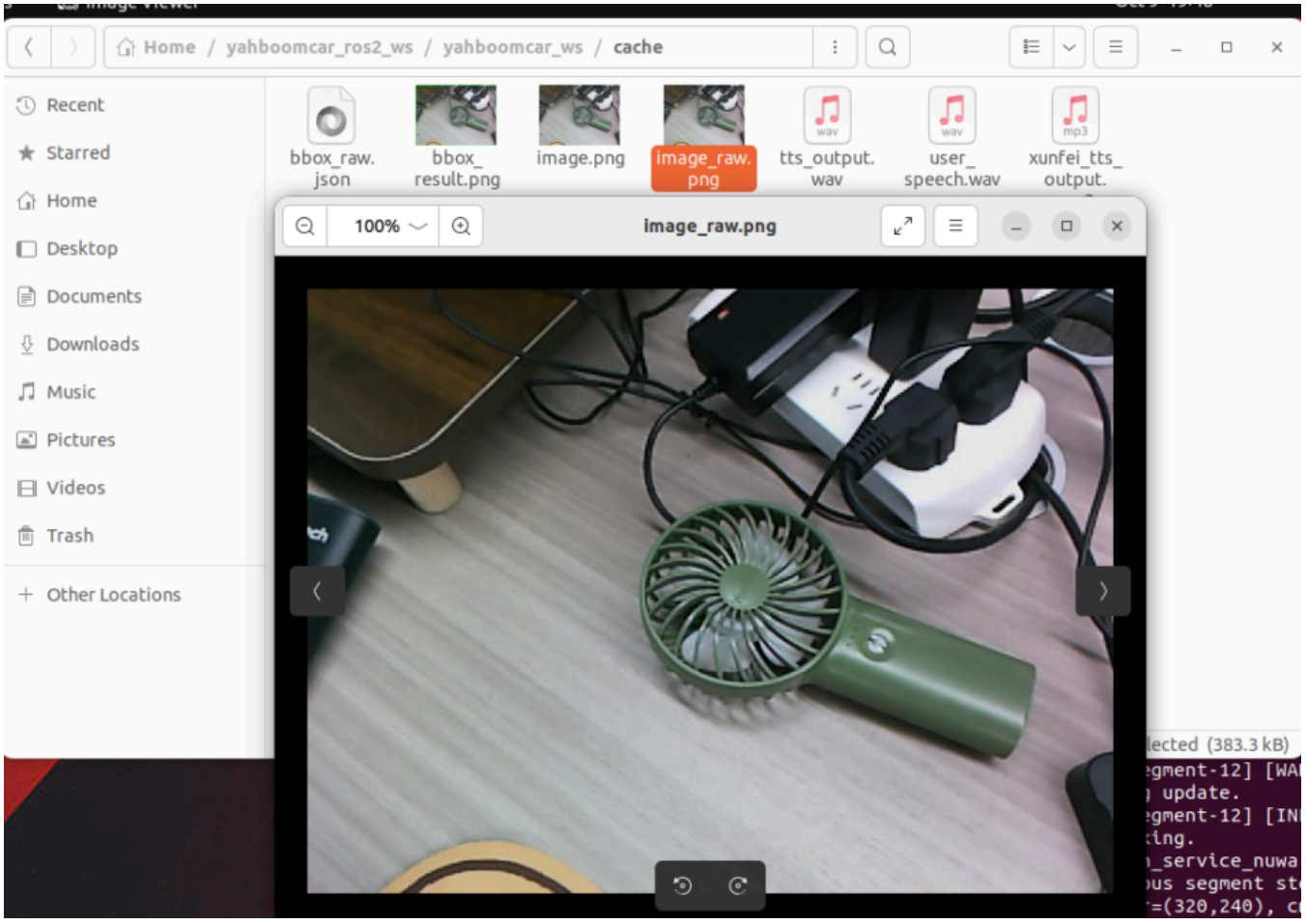
Enter the test statement in the terminal:

```
Please take a look at the items in front of you and tell me what each of them is used for.
```



For cases involving photo capture, view the captured photos in this path

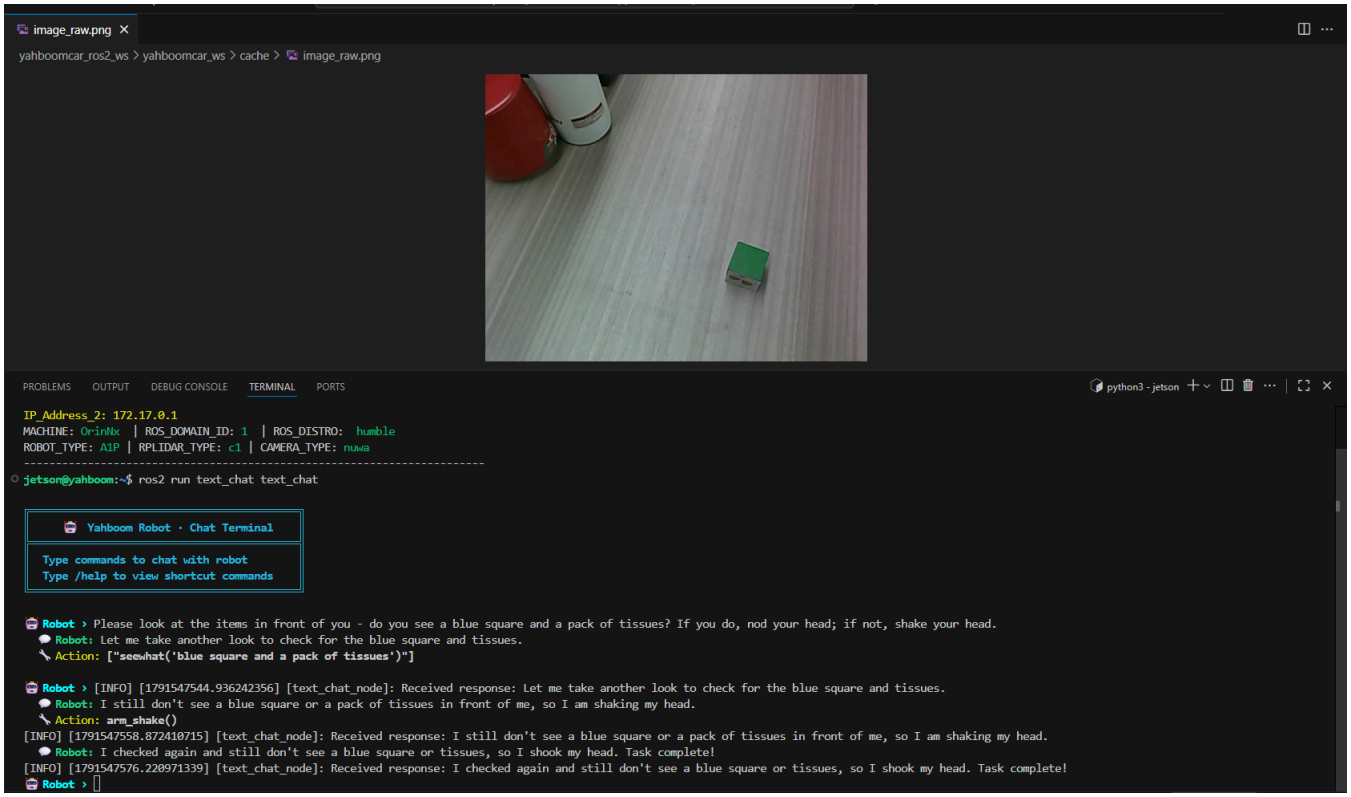
`~/yahboomcar_ros2_ws/yahboomcar_ws/cache/image_raw.png`



Case 2: Multi-Target Existence Judgment and Anthropomorphic Body-Language Feedback

Enter the test statement in the terminal:

```
Please look at the items in front of you - do you see a blue square and a pack of tissues? If you do, nod your head; if not, shake your head.
```



Execution Process and Behavior:

1. The robot calls `seewhat` to capture a photo of the current field of view;
2. The multimodal large model determines whether a "blue block" and a "pack of tissues" exist **at the same time** in the image;
3. If the condition holds, the large model issues the `arm_nod` (nod) action, and the robotic arm pitches twice in succession to nod; if the condition does not hold, it triggers the `arm_shake` (shake) action, and the base servo of the robotic arm swings left and right to signal "no".

4. Core Source Code Analysis

Source package path:
`~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains`

The system has built-in dual-scheme compatibility support for a **3D depth camera (Nuwa scheme)** and a **USB monocular camera (USB scheme)**. The master control launch script `llm_agent_control.launch.py` automatically loads the corresponding action service node according to the environment variable `CAMERA_TYPE` (default `nuwa`, optional `usb`):

- **3D depth camera scheme**: loads `action_service_nuwa.py`, which by default subscribes to `/ascamera_hp60c/camera_publisher/rgb0/image`;
- **USB camera scheme**: loads `action_service_usb.py`, which by default subscribes to `/usb_cam/image_raw`.

4.1 Visual Capture and Image Cache Processing (action_service_nuwa.py / action_service_usb.py)

In multimodal visual understanding tasks, the visual perception mechanisms of the two schemes are highly unified. When the large model needs to observe the surrounding environment, both call the `seewhat` action to capture the latest frame from the current camera Topic and store it in a normalized form:

```
def seewhat(self):
    """
    Save the current view image and feed the result back to the agent large model
    """
    if self.save_single_image():
        self._feedback(self.actionlog.get_text("image_feedback"))
        return None
    self._feedback("Image capture failed, grid image unavailable")
    return None
```

Inside the `save_single_image()` function, image resizing, grid drawing, and local caching are implemented:

```
def save_single_image(self, image_msg=None):
    """
    Save the 640x480 high-definition original image as well as a grid image with a 20px
coordinate ruler
    """
    image_msg = self.image_msg if image_msg is None else image_msg
    if image_msg is None:
        self.get_logger().warning("No camera image data has been received yet.")
        return False
    try:
        # Convert the ROS Image message to OpenCV BGR8 format
        cv_image = self.bridge.imgmsg_to_cv2(image_msg, "bgr8")
        if cv_image.shape[:2] != (480, 640):
            cv_image = cv2.resize(cv_image, (640, 480), interpolation=cv2.INTER_AREA)

        os.makedirs(self.image_cache_dir, exist_ok=True)
        # Draw the coordinate grid used for spatial reference
        gridded = self._draw_coordinate_grid(cv_image)

        # Save the high-definition original photo (for multimodal semantic recognition by
the large model and viewing by the user)
        self._write_cache_image(self.raw_image_cache_path, cv_image)
        # Save the grid photo (for localization reference)
        self._write_cache_image(self.image_cache_path, gridded)
        return True
    except Exception as e:
        self.get_logger().error(f"Image save exception: {e}")
        return False
```

- **Cache path**: the original photo is saved at `~/yahboomcar_ros2_ws/yahboomcar_ws/cache/image_raw.png`; the user can directly view the high-definition field-of-view image currently captured by the robot in this directory.

4.2 Multimodal Image Upload and Visual Question Answering (model_service.py)

In `model_service.py`, when a multimodal request is triggered, the node uploads the captured image to the Dify platform for deep semantic understanding:

```
# Multimodal vision request handling
if image_path is not None:
    upload_files = []
    # raw-picture lets the large model identify rich details such as the category, color,
and function of objects
    # grid-picture lets the large model reference the coordinate scale
    for name, path in (
        ("raw-picture", raw_image_path),
        ("grid-picture", image_path),
    ):
        if not path:
            continue
        with open(path, "rb") as file:
            response = self.client.file_upload("yahboom", {"file": (name, file,
"image/png")})
            response.raise_for_status()
            file_id = response.json().get("id")
            upload_files.append({"type": "image", "transfer_method": "local_file",
"upload_file_id": file_id})

    # Attach the image files to the question-answering request and send them to the
multimodal large model
    kwargs["files"] = upload_files
```

The large model combines the uploaded high-definition image with the user's natural-language question (such as "tell me what the objects are used for") to perform comprehensive reasoning and return a text answer.

4.3 Visual Conditional Judgment and Anthropomorphic Action Feedback (action_service_nuwa.py)

When the user requests "nod if present, shake if absent," after completing the visual judgment the large model outputs an `arm_nod` or `arm_shake` action, and the robotic arm performs intuitive body-language feedback:

```
def arm_nod(self):
    """ The robotic arm performs a nodding action """
    for i in range(3):
        if self.interrupt_event.is_set():
            self.pubSix_Arm(self.init_joints)
            return None
        # Pitch joint lowers the head
        tar_arm_joint = [90, 130, 0, 95, 90, 0]
        self.pubSix_Arm(tar_arm_joint)
        time.sleep(1.0)
        # Restore the initial posture
        self.pubSix_Arm(self.init_joints)
```

```
            time.sleep(1.0)
        return True

    def arm_shake(self):
        """ The robotic arm performs a shaking action """
        for i in range(3):
            if self.interrupt_event.is_set():
                self.pubSix_Arm(self.init_joints)
                return None
            # Base pan-tilt swings left
            tar_arm_joint = [140, 130, 0, 5, 90, 0]
            self.pubSix_Arm(tar_arm_joint)
            time.sleep(1.0)
            # Base pan-tilt swings right
            tar_arm_joint = [40, 130, 0, 5, 90, 0]
            self.pubSix_Arm(tar_arm_joint)
            time.sleep(1.0)
        self.pubSix_Arm(self.init_joints)
        return True
```

- **Action reset**: after the action is completed, the robotic arm automatically returns to the folded safe pose `init_joints`, ensuring it does not block the line of sight and keeps the body stable.